

Introduction

Hi, I'm Lokesh. I have around 3 years of experience as a Java Backend Developer, mainly working with Java, Spring Boot, and Microservices-based applications. Most of my experience is in the reinsurance domain, where I worked on scalable backend services, REST APIs, Kafka-based event-driven systems, and database optimization. I have hands-on experience with Java, Spring Boot, Kafka, MySQL, Redis, Docker, Kubernetes, Jenkins, and AWS. Currently, I work in an Agile environment where we follow sprint-based development and regular CI/CD production deployments.

Project Overview

In my current project, we are building a reinsurance platform that helps insurance companies transfer part of their risk to reinsurance companies to reduce financial exposure. Our application handles policy management, premium calculation, treaty management, claims processing, notifications, authentication, reporting, and communication with downstream systems.

The architecture is completely microservices-based, where each service has its own responsibility and separate database to avoid tight coupling. Services communicate using REST APIs for synchronous calls and Kafka for asynchronous event-driven communication.

For example, when a new policy is created, the Policy Service validates and stores the data in MySQL, then publishes an event to Kafka. Downstream services like Notification, Audit, and Reporting consume the event independently. This approach keeps services loosely coupled and improves system scalability and performance.

I mainly used Kafka for asynchronous processing, retry handling, and high-volume event processing such as claim updates, policy events, audit logs, and email notifications. For database optimization, I worked on query tuning, indexing, and pagination for large datasets. We also used Redis caching to improve API response time for frequently accessed data.

My responsibilities include developing REST APIs, implementing business logic, Kafka producer-consumer implementation, database optimization, bug fixing, production support, deployment activities, and code reviews. I was also involved in debugging production issues using logs and monitoring tools.

For deployment, we use Docker and Kubernetes. Each microservice has its own Docker image, and deployments are handled through Jenkins CI/CD pipelines.

I also worked on solving real-time challenges such as: handling concurrent updates using optimistic locking, managing timeout and retry mechanisms for slow services, improving reporting API performance using indexing and pagination, solving Redis cache consistency

issues using proper cache eviction strategies, and improving JWT token handling and refresh flow for authentication.

Overall, I enjoy backend development and building scalable distributed systems using microservices architecture. We are currently exploring Spring AI and MCP to make our platform smarter and more user-friendly. We are looking at using AI to assist in claims review, help business users generate reports using natural language, and expose our microservices as tools that an AI assistant can use. The main goal is to allow underwriters and analysts to interact with the system through simple conversational queries instead of depending completely on complex UI workflows.